

Documentación

Nombre del proyecto: SASS-ISO-II

Integrantes del equipo y roles Scrum:

Adrián Bohaben Torres (Scrum Master/Development Team)

Sergio Delgado Sanchez (Development Team)

Sergio Fernandez Del Río (Development Team)

Samantha Micaela Velasco Pinta (Development Team)

Índice

1 Diseño del Sistema

- a. Requisitos
- b. Arquitectura del proyecto
- c. Justificación de la arquitectura
- d. Diagramas del Sistema

2 Gestión de configuración

- a. Tecnologías utilizadas
- b. Guía de instalación y ejecución

3 Metodología de desarrollo: Scrum

- a. Principios del enfoque ágil
- b. Roles en Scrum
- c. Artefactos de Scrum
- d. Rituales Scrum realizados
- e. Historias de usuario y planificación ágil
- f. Planificación de los sprints
- g. Herramientas de apoyo

4 Funcionalidades implementadas

- a. Funcionalidades de usuario
- b. Funcionalidades técnicas
- c. Resultados obtenidos

1. Diseño del Sistema

a. Requisitos

Se pretende desarrollar un sistema similar a Airbnb donde personas particulares pueden reservar alojamientos de propietarios particulares.

- i. Tanto los inquilinos como propietarios deben registrarse previamente para utilizar el sistema.
- ii. Los propietarios (una vez logeados) pueden dar de alta propiedades (viviendas completas o habitaciones individuales) ofertándolas para ser alquiladas. Los propietarios marcan si desean que los usuarios realicen reservas directas, o guardarse el derecho de confirmar o no las solicitudes de reserva.
- iii. Los usuarios pueden primero buscar alojamiento (destino, fechas, tipo de inmueble, etc.) para examinar las opciones de alquiler. Excepcionalmente también pueden aplicar filtros avanzados para, por ejemplo: ver aquellos inmuebles con posibilidad de reserva inmediata; seleccionar filtros para ciertas comodidades; o seleccionar política de cancelación de reserva.
- iv. Para realizar la búsqueda de inmuebles no es necesario estar logeado, pero los usuarios registrados pueden agregar algunas propiedades a su lista de deseo.
- v. Los inquilinos logeados, una vez han seleccionado el inmueble a alquilar, pueden completar la reserva. Si la propiedad permite reserva inmediata se puede completar el pago directamente (vía tarjeta de crédito, débito o paypal).
- vi. Si la propiedad o inmueble no permite reserva inmediata se hace una solicitud de reserva (completando también en este momento el pago). La solicitud de reserva es notificada al propietario quién puede confirmar o no la reserva. Si la confirmación es positiva, esto se le notifica al inquilino. Si la solicitud es denegada, el sistema devuelve el dinero al inquilino y es igualmente notificado al cliente.

b. Arquitectura del proyecto

El proyecto sigue una arquitectura en capas o multicapa. Este tipo de arquitectura divide la aplicación en diferentes niveles con responsabilidades bien definidas, lo que facilita el mantenimiento, la escalabilidad y la reutilización del código.

i. Capas principales

Capa de Presentación (Vista o Interfaz de Usuario):

Es la parte del sistema con la que interactúa el usuario.

En nuestro caso, está formada por las ventanas gráficas o páginas que permiten realizar operaciones como registrarse, iniciar sesión, gestionar reservas o dar de alta un inmueble.

Capa de Negocio (Controller):

Aquí se implementan los gestores que procesan los datos y aplican las reglas

necesarias antes de interactuar con las entidades y base de datos (por ejemplo, GestorUsuarios, GestorInmuebles, GestorReservas).

Capa de Negocio (Dominio) :

Representa los objetos o entidades principales del sistema (por ejemplo, Usuario, Inmueble, Reserva, SolicitudReserva).

Capa de Persistencia:

Es la encargada de **acceder y gestionar la base de datos..**

ii. Justificación de la arquitectura

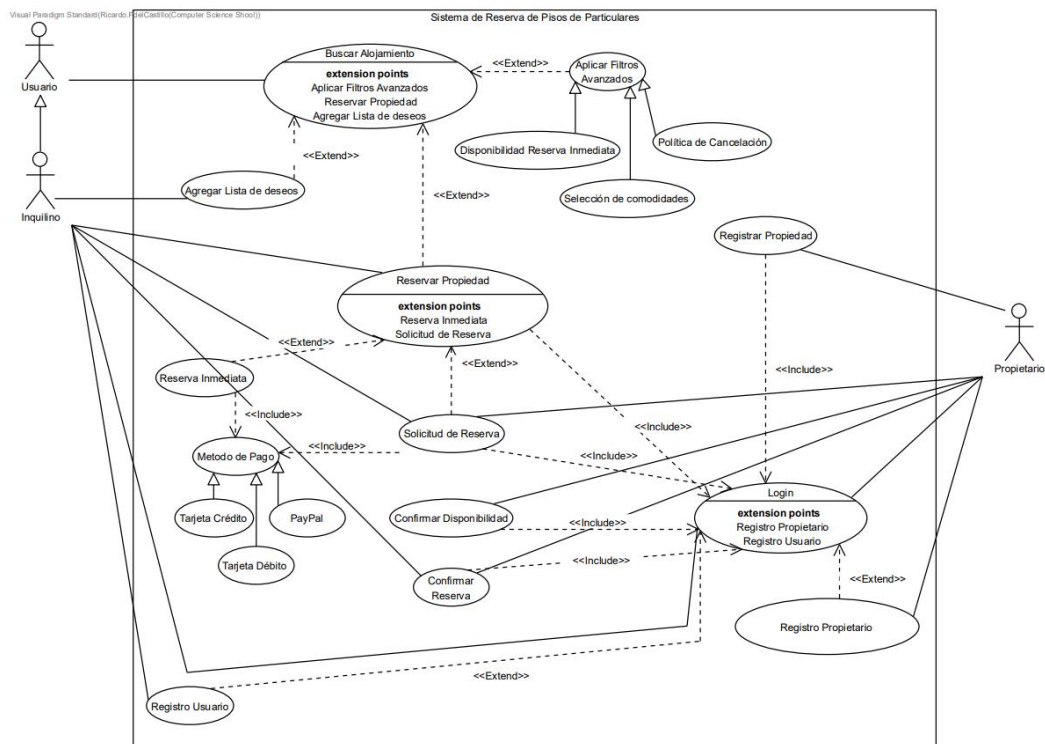
Esta organización en capas permite separar las responsabilidades, lo que facilita el mantenimiento y la comprensión del código.

La capa de persistencia está aislada, de forma que en el futuro podría cambiarse el tipo de base de datos (por ejemplo, de Derby a MySQL) sin modificar la lógica de negocio.

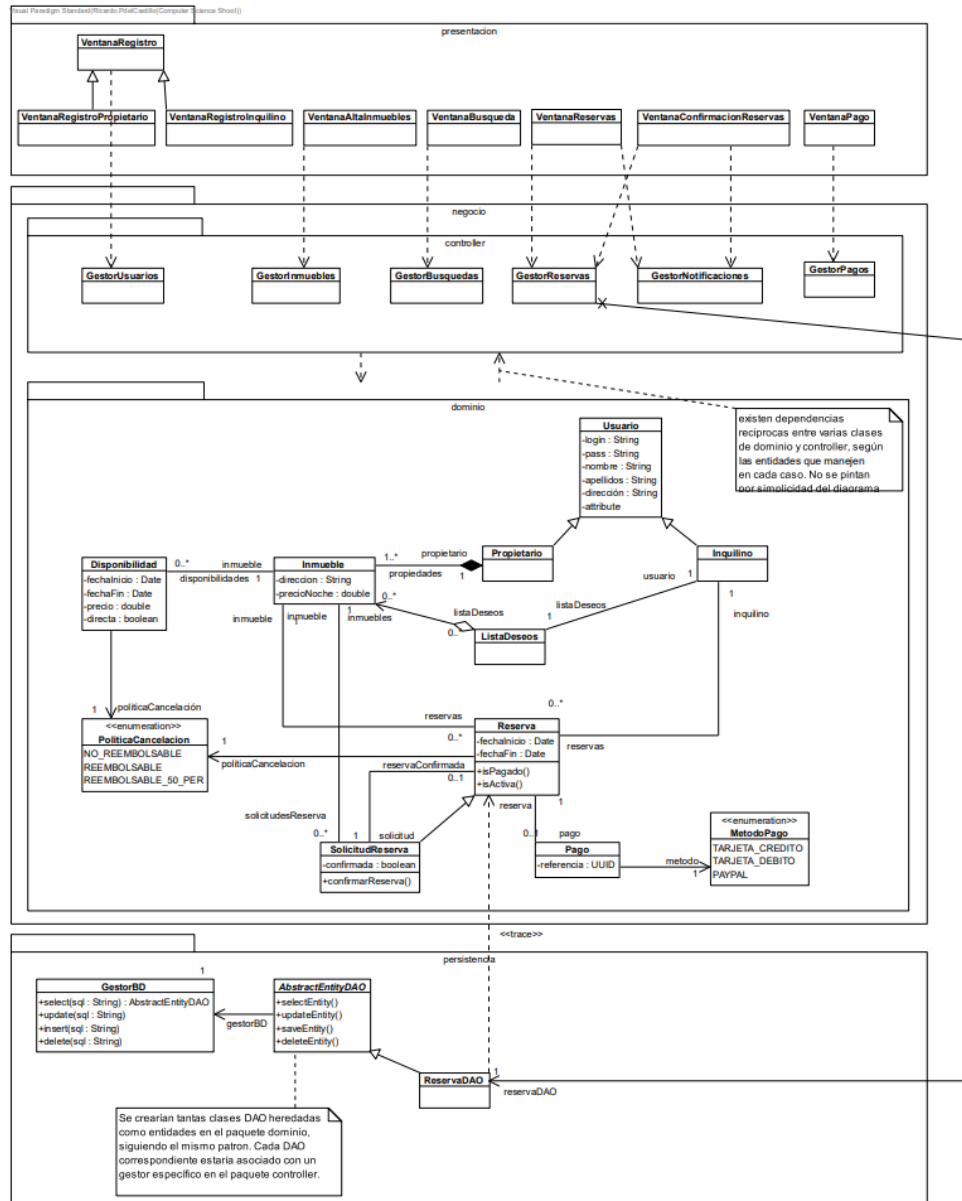
La capa de presentación se mantiene independiente, permitiendo sustituir la interfaz (por ejemplo, pasar de una interfaz gráfica local a una aplicación web) sin afectar el resto del sistema.

c. Diagramas del Sistema

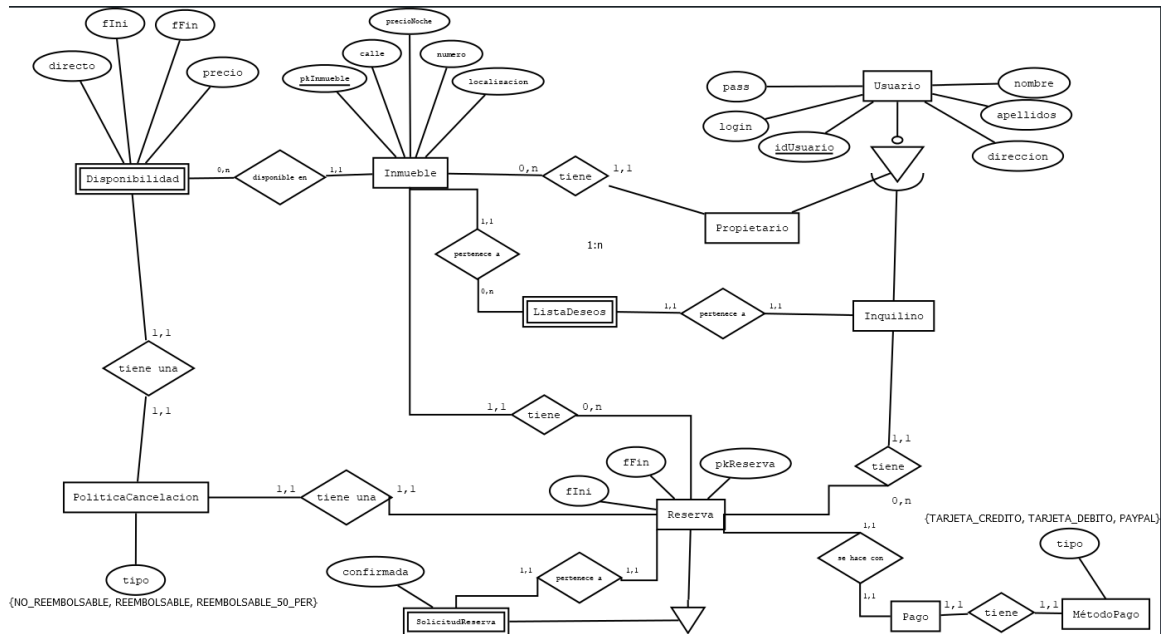
i. Casos de uso



ii. Diagrama de clases



iii. Diagrama entidad-relacion



2. Gestion de configuración

a. Tecnologías utilizadas

Eclipse: Es el entorno de desarrollo integrado (IDE) elegido para este proyecto porque ofrece una integración completa con **Java** y **Maven**, lo que facilita la compilación, ejecución del código. Además, permite gestionar el proyecto de forma estructurada y visual, y es uno de los recomendados por los profesores.

Maven: Se utiliza como herramienta de gestión de dependencias y automatización de compilaciones. Gracias a Maven, es posible manejar las librerías necesarias (por ejemplo, Spring Boot, Derby o JPA) y garantizar que todos los miembros del equipo tengan el entorno de trabajo configurado de la misma forma, lo que mejora la coherencia del proyecto.

Spring Boot: Es el framework principal del proyecto. Permite desarrollar aplicaciones Java estructuradas bajo el patrón MVC (Modelo–Vista–Controlador) y facilita la creación de servicios REST. Su uso reduce significativamente configuración manual y mejora la modularidad y escalabilidad del sistema.

Apache Derby: Se ha elegido como base de datos relacional embebida por su facilidad de uso y su integración directa con proyectos Java. Al ser ligera y no requerir instalación adicional, resulta ideal para entornos de desarrollo y pruebas, permitiendo que cada miembro del equipo trabaje con su propia base de datos local de manera sencilla. Se habían valorado el uso de MySQL, o H2 entre otras, pero nos hemos decantado por apache al final.

DBeaver: Es una herramienta de administración de bases de datos que permite conectarse a Derby y visualizar sus tablas de forma gráfica. Se utiliza para realizar consultas SQL sin necesidad de hacerlo desde código, lo que facilita las tareas de desarrollo y depuración.

b. Guía de instalación y ejecución

Requisitos previos:

Herramienta	Versión recomendada	Descripción
Java JDK	21 o superior	Necesario para ejecutar Spring Boot
Maven	3.9	Sistema de gestión y compilación del proyecto
Git	Última versión	Para clonar y gestionar el repositorio
Eclipse IDE for Java Developers	2023-09 o superior	Entorno de desarrollo integrado
Spring Boot	3.x	Framework principal del proyecto
DBeaver	Última versión	Cliente SQL para gestionar la base de datos
PostgreSQL / MySQL	-	Motor de base de datos utilizado

Pasos para ejecutar el proyecto (Eclipse):

1. Clonar el repositorio con los siguientes comandos:

```
git clone https://github.com/SASS-ISO-II/AV.git
```

```
cd AV
```

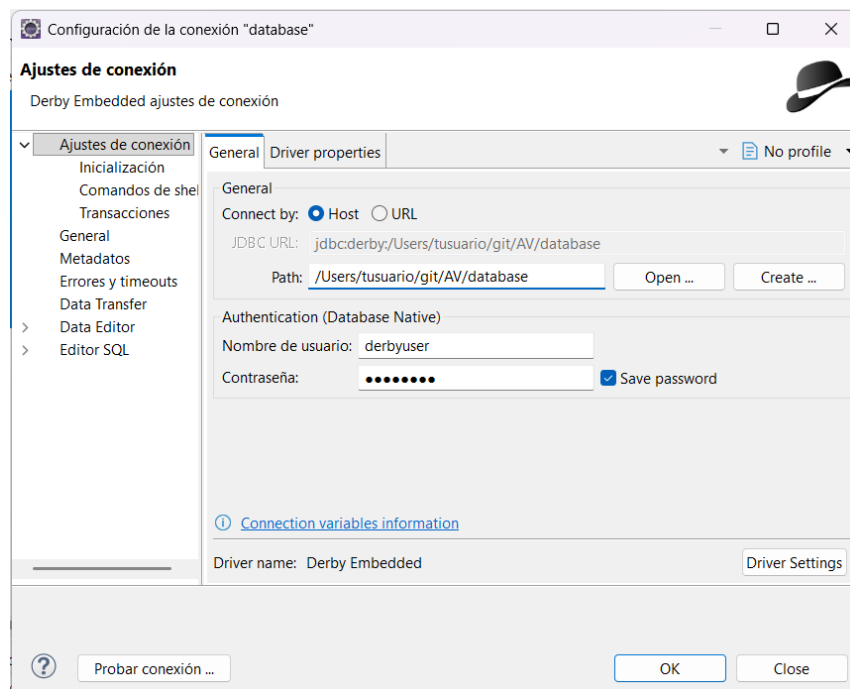
2. Verificar herramientas mencionadas anteriormente:

```
java -version
```

```
mvn -version
```

```
git -version
```

3. Cambiar la ruta de la carpeta database en la interfaz de dbeaver de eclipse(contraseña :password)



4. Compilar el proyecto.

```
mvn clean install
```

5. Ejecutar aplicación:

mvn spring-boot:run

6. Acceder desde el navegador

<http://localhost:8081/inicio>

3. Metodología de desarrollo: Scrum

El proyecto se ha desarrollado siguiendo la metodología **Scrum**, un framework ágil orientado a la **gestión de proyectos iterativos e incrementales**, cuyo objetivo es ofrecer valor al cliente de forma continua mediante entregas frecuentes de producto funcional.

Scrum fomenta la **colaboración, adaptación al cambio y mejora continua** del proceso de desarrollo.

a. Principios del enfoque ágil

Scrum se basa en los valores del **Manifiesto Ágil**, que prioriza:

- **Individuos e interacciones** sobre procesos y herramientas.
- **Software funcionando** sobre documentación extensiva.
- **Colaboración con el cliente** sobre negociación contractual.
- **Respuesta al cambio** sobre seguir un plan rígido.

Estos principios garantizan que el equipo se centre en entregar valor real al usuario y adaptarse a los cambios durante el desarrollo.

b. Roles en Scrum

En nuestro equipo se definieron los siguientes **roles Scrum**:

- **Scrum Master:** *Adrián Bohaben Tores* — Responsable de asegurar la correcta aplicación de la metodología y facilitar la comunicación del equipo.
- **Development Team:** *Sergio Delgado Sánchez, Sergio Fernández del Río y Samantha Micaela Velasco Pinta* — Equipo encargado del desarrollo técnico y funcional de la aplicación, trabajando de forma organizada.
- **Product Owner:** rol asumido de forma compartida entre el equipo, responsable de priorizar las tareas y validar las funcionalidades del producto en GitHub Projects.

c. Artefactos de Scrum

Durante el desarrollo se emplearon los tres artefactos principales del framework:

- **Product Backlog:** lista de historias de usuario mantenida en **GitHub Projects**. Contiene todas las funcionalidades deseadas (registro de usuarios, gestión de inmuebles, reservas, etc.).
- **Sprint Backlog:** conjunto de tareas seleccionadas del Product Backlog para desarrollarse en cada sprint. Cada tarea se movía en el tablero Kanban (*In Progress* → *In Review* → *Done*).
- **Incremento:** versión funcional del sistema generada al final de cada sprint, lista para ser mostrada y validada.

d. Rituales Scrum realizados

A lo largo del proyecto se llevaron a cabo los principales **eventos Scrum**, siguiendo la estructura propuesta por el framework:

- **Sprint Planning:** planificación de las historias de usuario a desarrollar en cada sprint. Se definían los objetivos principales y se estimaban los esfuerzos de desarrollo.
- **Daily Scrum:** reuniones breves de seguimiento entre los miembros del equipo (coordinadas vía GitHub y mensajería).
- **Sprint Review:** revisión y demostración de las funcionalidades terminadas al finalizar cada sprint.
- **Sprint Retrospective:** análisis conjunto de los resultados obtenidos y definición de mejoras para el siguiente sprint.

e. Historias de usuario y planificación ágil

Las **historias de usuario** son la unidad básica de trabajo, redactadas con el formato:

“Como [rol del usuario], quiero [acción o funcionalidad], para [objetivo o beneficio].”

Ejemplos:

- Como propietario, quiero **dar de alta un inmueble**, para poder ofrecerlo en alquiler.

- Como inquilino, quiero **reservar un alojamiento**, para asegurar mi estancia en las fechas deseadas.

La planificación se realizó de manera **incremental**, incorporando nuevas historias en cada sprint en función de la retroalimentación y los avances del equipo.

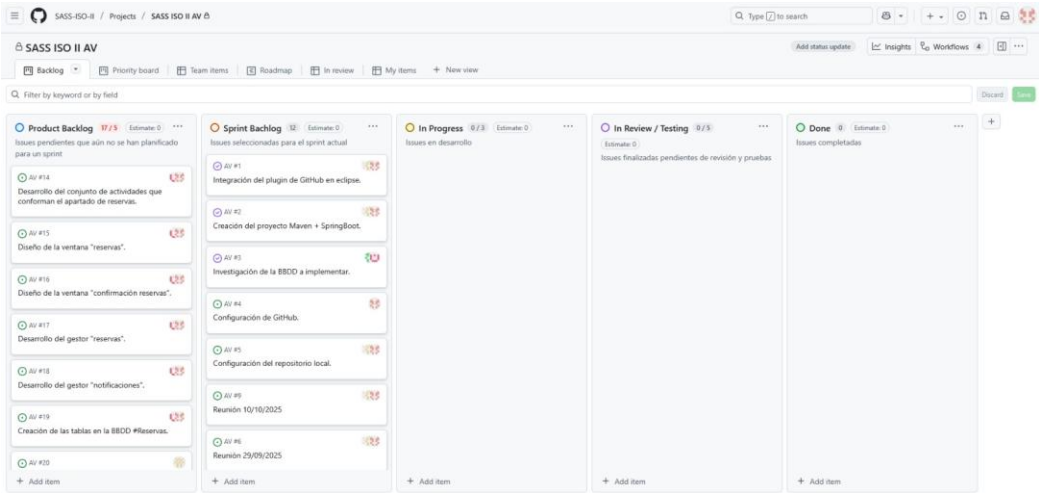
f. Planificación de los sprints

El trabajo se organizó en **tres sprints**, cada uno con una duración aproximada de **dos semanas**, lo que permitió entregar incrementos funcionales al final de cada ciclo.

Sprint	Duración	Objetivos principales	Historias de usuario	Resultado
Sprint 1	1 semanas	Configuración del entorno y base del backend	Registro y login de usuarios	Sistema con autenticación básica funcional
Sprint 2	1 semanas	Implementación de gestión de inmuebles y reservas	Alta y listado de inmuebles, gestión de reservas	Backend funcional y conexión a Derby
Sprint 3	1 semanas	Pruebas, integración y refinamiento de la interfaz	Validación de datos, mejora UI	Versión estable del producto mínimo viable

Capturas de cada sprint en github:

Sprint 1:



Sprint 2:

SASS-ISO-II / Projects / SASS ISO II AV

Backlog

Priority board

Team items

Roadmap

In review

My items

New view

Filter by keyword or by field

Discard

Save

Product Backlog 0/5 Estimate: 0

Issues pendientes que aún no se han planificado para un sprint

Sprint Backlog 5 Estimate: 0

Issues seleccionadas para el sprint actual

AV #10

Reunión 13/10/2025

AV #14

Desarrollo del conjunto de actividades que conforman el apartado de reservas.

AV #20

Desarrollo del conjunto de actividades que conforman el apartado de login.

AV #27

Desarrollo del conjunto de actividades que conforman el apartado de alta inmuebles.

AV #28

Ampliación de la documentación del proyecto.

In Progress 13/3 Estimate: 0

Issues en desarrollo

AV #15

Diseño de la ventana "reservas".

AV #16

Diseño de la ventana "confirmación reservas".

AV #17

Desarrollo del gestor "reservas".

AV #18

Desarrollo del gestor "notificaciones".

AV #19

Creación de las tablas en la BBDD #Reservas.

AV #21

Diseño de la ventana "registro".

AV #22

Diseño de la ventana "registro propietarios"

In Review / Testing 1/5 Estimate: 0

Issues finalizadas pendientes de revisión y pruebas

AV #12

Integración de la documentación del proyecto.

Done 10 Estimate: 0

Issues completadas

AV #1

Integración del plugin de GitHub en eclipse.

AV #2

Creación del proyecto Maven + SpringBoot.

AV #3

Investigación de la BBDD a implementar.

AV #4

Configuración de GitHub.

AV #5

Configuración del repositorio local.

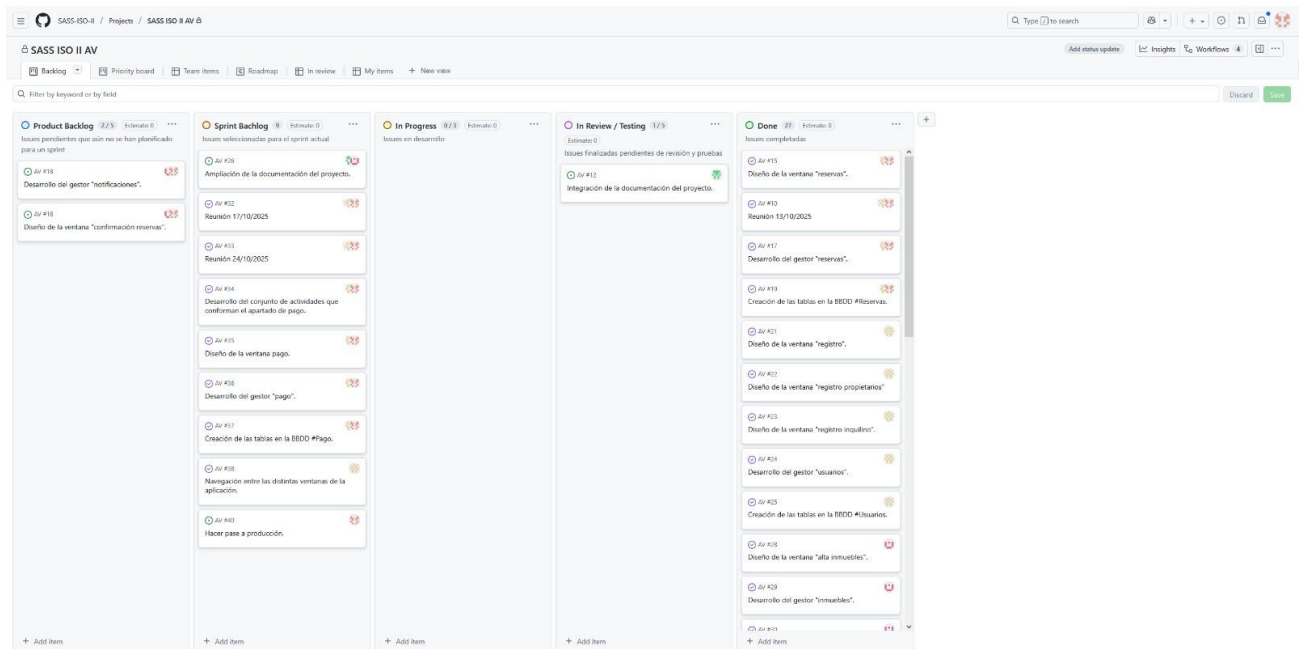
AV #9

Reunión 10/10/2025

AV #6

Reunión 29/09/2025

Sprint 3:



g. Herramientas de apoyo

Para la gestión del proyecto y la coordinación del equipo se utilizaron:

- **GitHub Projects** → tablero Kanban para la planificación y seguimiento del trabajo.
- **GitHub Issues** → gestión de historias de usuario y registro de tareas.
- **Eclipse** → entorno de desarrollo principal.
- **Maven y Spring Boot** → automatización, compilación y ejecución del proyecto.
- **Apache Derby y DBeaver** → gestión de la base de datos embebida.

4. Funcionalidades implementadas

El **Producto Mínimo Viable** desarrollado incluye las principales funcionalidades necesarias para garantizar el funcionamiento básico de la aplicación, tanto desde la perspectiva del **usuario final** como desde el **entorno técnico**.

A continuación, se detallan las funcionalidades implementadas:

a. Funcionalidades de usuario

Registro y autenticación de usuarios

- Permite registrar a los **inquilinos y propietarios**, almacenando sus datos en la base de datos Derby.
- El sistema incluye **inicio de sesión (login)** y validación de credenciales, asegurando el acceso a las funcionalidades según el tipo de usuario.

Alta de inmuebles (propietario)

- Los propietarios pueden **dar de alta nuevos inmuebles** mediante un formulario, introduciendo información como descripción, ubicación, tipo de alojamiento o precio.
- Cada inmueble queda asociado al propietario que lo registró.

Consulta y listado de inmuebles (inquilino)

- Los usuarios pueden **consultar el listado completo de inmuebles** disponibles en el sistema.
- La interfaz permite visualizar información relevante de cada propiedad (nombre, ubicación, tipo, disponibilidad, etc.).
- Se ha implementado una **navegación fluida entre vistas**, permitiendo pasar del listado a la ficha del inmueble o al formulario de reserva.

Reserva de inmuebles

- Los inquilinos logeados pueden **reservar un inmueble** seleccionado.
- El sistema registra la reserva en la base de datos, vinculando al usuario y al inmueble correspondiente.

b. Funcionalidades técnicas

1. Persistencia de datos (Base de datos Derby)

- Todos los datos (usuarios, inmuebles, reservas) se almacenan de forma **persistente** en una base de datos **Apache Derby** embebida.

2. Arquitectura en capas (Spring Boot)

- La aplicación sigue la estructura **MVC (Modelo-Vista-Controlador)** propia de Spring Boot:

- *Controladores* gestionan las peticiones del usuario.
- *Servicios* implementan la lógica de negocio.
- *Repositorios* gestionan la comunicación con la base de datos.
- *Vistas (Thymeleaf)* renderizan las páginas HTML dinámicas.

3. Navegación entre vistas

- Se ha diseñado una **navegación coherente** entre pantallas.

4. Gestión de sesiones

- El sistema mantiene la sesión del usuario logeado, permitiendo hacer funcionalidades específicas (por ejemplo, alta de inmuebles solo disponible para propietarios).

5. Validación y manejo de errores

- Se han implementado **mensajes de error y confirmación** para mejorar la usabilidad.

c. Resultados obtenidos

- Se ha logrado una **versión funcional del sistema** que permite completar el ciclo básico de uso tanto para propietarios como par inquilinos:

Propietarios:

registrarse → iniciar sesión → dar de alta un inmueble.

Inquilinos:

registrarse → iniciar sesión → ver lista de inmuebles → → realizar una reserva.

- La aplicación dispone de **persistencia completa** de la información y una **navegación estable y fluida** entre las distintas vistas.